

Microservices Architecture – 30 Interview Questions with Detailed Answers

1. What is a Microservice?

A microservice is a small, independently deployable service that focuses on a single business capability. It communicates with other services over lightweight protocols (HTTP/gRPC/messaging) and can be developed, deployed, and scaled independently.

2. Advantages of Microservices over Monolithic architecture

Independent scalability, faster deployments, technology flexibility, better fault isolation, and team autonomy.

3. Disadvantages of Microservice architecture

Increased operational complexity, network latency, distributed debugging, data consistency challenges, and higher infra cost.

4. When should you not use Microservices?

For small teams, simple applications, unclear domain boundaries, or when operational maturity (DevOps, monitoring) is missing.

5. Design principles of Microservices

Single Responsibility, loose coupling, high cohesion, decentralized data management, automation, and observability.

6. What is the 12-Factor methodology?

A set of principles for building cloud-native apps: codebase, dependencies, config, backing services, stateless processes, port binding, concurrency, disposability, dev-prod parity, logs, admin processes.

7. Why are Microservices stateless?

Stateless services scale easily, recover faster from failures, and simplify load balancing. State is externalized to DB/cache.

8. Why Spring Boot + Spring Cloud for Microservices?

Spring Boot simplifies service creation, while Spring Cloud provides service discovery, config management, circuit breakers, and distributed tracing support.

9. Should multiple Microservices share a database?

No. Each service should own its data to avoid tight coupling and allow independent evolution.

10. What is Distributed Tracing and why is it needed?

It tracks a request across multiple services to identify latency issues, failures, and bottlenecks.

11. How is Distributed Tracing implemented?

Using trace IDs propagated across services with tools like Zipkin, Jaeger, and OpenTelemetry.

12. How do Microservices communicate internally and externally?

Internally via REST/gRPC/messaging, externally via API Gateway.

13. Which Microservice design patterns have you used and why?

API Gateway, Circuit Breaker, SAGA, CQRS, Service Discovery for scalability and fault tolerance.

14. Database design patterns in Microservices

Database per Service, Event Sourcing, CQRS.

15. What is the SAGA pattern?

A pattern for managing distributed transactions using a sequence of local transactions with compensating actions.

16. Explain CQRS

Command Query Responsibility Segregation separates read and write models for better scalability and performance.

17. Best patterns for read-heavy vs write-heavy systems

Read-heavy: CQRS + caching. Write-heavy: Event-driven architecture.

18. Choreography vs Orchestration

Choreography uses events without a central coordinator; Orchestration uses a central controller.

19. Fault tolerance mechanisms in Spring Microservices

Retry, Circuit Breaker, Rate Limiting, Timeouts, Bulkheads.

20. Circuit Breaker pattern with examples

Prevents cascading failures by stopping calls to failing services (e.g., Resilience4j CircuitBreaker).

21. Spring Boot annotations for Circuit Breaker

@CircuitBreaker, @Retry, @RateLimiter using Resilience4j.

22. Libraries used for Circuit Breaker implementation

Resilience4j (preferred), Hystrix (deprecated).

23. How to call methods asynchronously in Spring

Using @Async with CompletableFuture or Executor.

24. How to call another Microservice asynchronously

Using messaging systems like Kafka/RabbitMQ or async HTTP clients.

25. Communication styles between Microservices

Synchronous (REST/gRPC) and Asynchronous (events/messages).

26. How to restrict one Microservice from calling another

Using authentication, authorization, API Gateway rules, network policies.

27. How to securely store credentials in Spring Boot Microservices

Using Vault, Kubernetes Secrets, environment variables, or encrypted config.

28. What is Service Discovery and why is it required?

Automatically locates services dynamically (e.g., Eureka, Consul).

29. How do you handle configuration management in Microservices?

Centralized config using Spring Cloud Config, Consul, or Kubernetes ConfigMaps.

30. How do you version Microservice APIs?

URI versioning (/v1), header-based versioning, or backward-compatible changes.